

Game Development Report

1. Introduction

I have developed a 2D Top-Down Dungeon Crawler game with local 2-player Cooperative gameplay. My game is implemented entirely in Java and uses the Swing library. My game follows the Model View Controller pattern. The main idea of my game is to survive waves of enemies through the usage of movement and combat mechanics and progressing through levels of the dungeon. My game has diverged significantly from the original basic template. I have implemented custom sprites, a wave-based level system, custom overlapping hitboxes, custom player controls, difficulty selection, and save states. I have included 2-player Cooperative gameplay on the same screen, which showcases advanced usage of simultaneous Keyboard IO masking.

2. Gameplay Mechanics and Features

2.1 Dual Player Co-Op System

A major feature of this game is the implementation of real local multiplayer.

- Player 1 - Red Knight
The character is controlled using the WASD keys for all directions. Fights are started using the SPACE key or by using a complex Mouse Click Listener to deploy an Orbital Spinning Sword.
- Player 2 - Blue Knight
The character is controlled using the Arrow Keys. Fights are started using the ENTER key. In order to avoid the common "Diagonal Speed Glitch" that is commonly encountered in simple 2D games, extreme care is taken to use Vector Normalization in the bounding box physics to avoid the character moving at a faster rate than normal when two keys are pressed at a time.

2.2 Difficulty Scaling and Wave Progression

When the player selects the option for a 'New Game,' they are given three difficulty options: Normal, Hard, and Nightmare, which are buttons that serve as a configuration state for the game, controlling the global speed multipliers of the game engine and the spawn rate of the enemies. For the horizontal nature of the game, the game uses a wave system. There is a quota of enemies, such as Goblins and Skeletons, that spawn off the screen, avoiding "popping," and once the player has cleared the quota of enemies for the current floor, a 'Door' object, or the 'Level Hole,' appears gracefully, wiping the current board clean and moving the players through the next floor, effectively increasing the spawn limits of the enemies and the speed of the enemies' attacks.

2.3 Combat, Scoring, and Consumables

The fighting mechanism involves a piercing "orbital melee" system. When the player attempts to attack, an offset "blade sprite" is created and uses rotational vector math to orbit around the player. Enemies use dynamic code to find the closest player using distance formulas ($\text{Math.sqrt}(dx * dx + dy * dy)$) and a separation algorithm to prevent clusters. Killing enemies increases the player's score. Additionally, an RNG (Random Number Generator) system for enemy deaths provides a 15% chance to spawn a Health Potion, which replenishes lost lives upon contact.

3. Technical Implementation and MVC Architecture

3.1 The Model (Model.java)

All the mathematical state changes occur without any visualization through the Model. It controls the CopyOnWriteArrayList data structure for all active objects (Enemies, Bullets, Potions). This thread-safe list eliminates any possibility of a concurrent modification exception during the addition or removal of objects during the frame. The collision detection is fully encapsulated within the Model using AABB (Axis-Aligned Bounding Box) rectangular collision detection based on the X and Y coordinates offset by the GameObjects' hitbox.

3.2 The View (Viewer.java and MainWindow.java)

Viewer.java extends JPanel and represents the immediate canvas for drawing. This class only uses the Model class's getter methods to retrieve information and uses the Graphics2D API to draw graphics. Images are loaded centrally to improve performance and are rendered using delta time. MainWindow.java, This class represents the JFrame and initializes the UI start screens and HUD overlays natively, rather than using code to render buttons onto the in-game canvas.

3.3 The Controller (Controller.java)

The Controller class, built on the Singleton design pattern, safely catches Java AWT events and translates these events to raw boolean values (`valuesisKeyWPressed`, `isMousePressed`). Unlike the asynchronous interrupt calls from external events in the traditional model, the Model class now polls this controller once per frame to maintain the integrity of the game loop.

4. Graphics and Asset Management

Audio visual fidelity was an important factor. The template graphics for the generic template graphics set were replaced with completely custom dungeon aesthetic pixel art. Care was taken to ensure that these changes didn't cause memory issues, and `ImageIO.read()` is safely leveraged to buffer the sprites. The `startscreen.png` scales dynamically using `Image.SCALE_SMOOTH` to fit the boundary properly. The sprites had transparent alpha channels, allowing for proper overlapping without the harsh square borders interfering with the background map.

5. Advanced Extensions

- **State Persistence (Save / Load):** Native file parsing system using standard Java PrintWriter and BufferedReader. At any time, the user can press F5 to serialize their current state of Level, Score, Lives, Difficulty Multipliers, and Quotas directly to a localized savegame.txt file. They can then load this state in perfectly from the front menu screen or using F9.
- **Swarm AI Separation:** Rather than having the enemies simply follow the player and stack on top of each other into an indistinguishable mass, the enemy pathfinding code uses a vector "push" radius. When two goblins get too close to each other, their vector push separates them slightly, creating a very realistic "horde" effect of surrounding the player.

6. Conclusion

This creates a complete, stable, and entertaining 2D Action RPG framework. By using class separation principles and effective Java Swing thread management, the game is able to hold a firm frame per second loop with dozens of active on-screen entities, projectiles, commands, and pixel-level collision detection. The addition of the dynamic Save feature, Co-Op processing, and physics is a direct answer and extension of the core principles learned throughout the module.